Codelab Formatting Guide

Bug or feature request? File it here.

Getting Started

Copy this template doc as a starting point and then iteratively mutate the metadata and contents to your liking, following the formatting rules described below.

To preview a codelab:

- (optional one-time setup) Install the Preview Codelab Chrome extension in your browser.
- Click the Chrome extension's button while you're in your codelab Google Doc tab, or
- Navigate manually to **https://codelabs-preview.appspot.com/?file_id=[google-doc-id]**.

Formatting Reference

1. Table of Contents

   Every codelab document must use the **Heading 1** paragraph style to delineate the steps of the codelab. In turn, these headings will be used to automatically create a Table of Contents which shows the student exactly where they are in the codelab and lets them jump to any step.

   The table of contents disappears for smaller browsers but is still available from the hamburger menu.

2. Codelab Metadata (Google Docs)

   There is some additional metadata that is required in order to properly publish a codelab. This metadata should be added as a **two-column table** anywhere before the first step of the codelab. For example:

   You are free to add your own metadata here if you'd like but certain key/value pairs are reserved for specific codelab publishing features. The current list of reserved metadata terms are:

   - **Summary:** A short summary of the codelab that will be shown in the codelab browser UI.
   - **URL:** The URL suffix where this codelab will be published, i.e. the path after the root URL to a collection of codelabs. For example, the Google Codelabs site uses

codelabs.developers.google.com/codelabs/ as root and this value is appended to that base to form the unique URL for a given codelab.
    - **Category:** A single, top-level category that will be used to group codelabs by platform. Categories are normally curated by an organization (e.g. we have a set we use for the Google Codelabs site) but each publisher is free to use this value at their discretion.
    - **Environment**: A tag that allows use to output some codelabs for a specific environment. All codelabs default to the "Web" environment but given some hardware constraints we might only want to generate them for a "Kiosk" environment where we know people will have the right hardware. \

You can also use this to target specific events, for instance:
"Web, polymer-summit" (without quotes) * **Status:** One or more of (Draft, Published, Deprecated, Hidden) to indicate the progress and whether the codelab is ready to be published. 'Hidden' implies the codelab is for restricted use, should be available only by direct URL, and should not appear on the main index page. * **Feedback Link:** The URL that the student should be sent to when they click on the feedback link to report a bug in the codelab. * **Analytics Account:** This allows you to specify a custom Google Analytics ID for your codelab. If no ID is specified, it defaults to a global codelabs analytics account.

1. Codelab Metadata (Markdown)

    You are free to add your own metadata here if you'd like but certain key/value pairs are reserved for specific codelab publishing features. The current list of reserved metadata terms are:

    - **summary:** A short summary of the codelab that will be shown in the codelab browser UI.
    - **id:** The name of the folder that will be generated once you export the markdown file via claat.
    - **categories:** A single, top-level category that will be used to group codelabs by platform. Categories are normally curated by an organization (e.g. we have a set we use for the Google Codelabs site) but each publisher is free to use this value at their discretion.
    - **environments**: A tag that allows use to output some codelabs for a specific environment. All codelabs default to the "Web" environment but given some hardware constraints we might only want to generate them for a "Kiosk" environment where we know people will have the right hardware. \

You can also use this to target specific events, for instance:
"Web, polymer-summit" (without quotes) * **status:** One or more of (Draft, Published, Deprecated, Hidden) to indicate the progress and whether the codelab is ready to be published. 'Hidden' implies the codelab is for restricted use, should be available only by direct

URL, and should not appear on the main index page. * **feedback link:** The URL that the student should be sent to when they click on the feedback link to report a bug in the codelab. * **analytics account:** This allows you to specify a custom Google Analytics ID for your codelab. If no ID is specified, it defaults to a global codelabs analytics account. * **tags:** Add relevant tags to make your codelab easily found. * **authors:** Indicate the author(s) of this specific codelab.

1. Headers

   Within the steps of your codelab you should use the **Heading 2**, **Heading 3** and **Heading 4** paragraph styles to organize your content. These will get translated to `<h2>`, `<h3>` and `<h4>` tags in the codelab markup.

   Also, if you wish to include some additional administrative information at the beginning of your codelab you can freely use the **Heading 2**, **Heading 3** and **Heading 4** paragraph styles and they will not show up in the codelab as long as they appear before the first **Heading 1**.

2. Text Styles

   For the most part, it doesn't matter which fonts you use in your Google Doc draft; everything will be formatted using Roboto in the final codelab.

   There are some simple ways that you can add emphasis to certain parts of the text. Bolded and italicized text will be passed through to the codelab markup as `<strong>` and `<em>` tags respectively. Also, passages of text formatted with the `Courier New` font will be passed through as an inline `<code>` tag in the markup.

3. Responsive Images

   Inline images in your codelab should just work seamlessly. You can re-size them in your codelab document and that width will be applied as a **max-width** on the image in the codelab markup so that images are the same size relative to the text but also scale down appropriately for smaller browsers.

4. Youtube Videos

   Youtube Videos can be embedded by doing:

   - Add an image in the document. The image can be a screenshot of the video for instance but it doesn't really matter since it won't be displayed but replaced by the embedded video.
   - Add an "Alt Text" to the image by doing **Cmd+Opt+Y** or **Right click > "Alt Text..."**
   - Put a Youtube video link in the **Description** field of the Alt Text. in the format `https://www.youtube.com/watch?v=[video_ID]`

> Specifying a start time is not supported at this time.

5. Embedded Iframes

   Iframes can be embedded by doing:

   - Add an image in the document. The image can be a screenshot of the iframe for instance but it doesn't really matter since it won't be displayed but replaced by the embedded iframe.
   - Add an "Alt Text" to the image by doing **Cmd+Opt+Y** or **Right click > "Alt Text..."**
   - Put a full URL in the **Description** field of the Alt Text. in the format `https://www.domain.com/watch?foo=bar`. Note that for security reasons, iframe embbedding is limited to an enumerated set of allowable iframe source URLs. Feel free to submit a PR if you'd like to augment that list or tweak your own version of the claat command.

6. Info Boxes

   For additional information that you would like to specially call-out in your codelab, there are two styles of info boxes:

   1. Positive information like best practices and time saving tips should be formatted as a **single-cell table** with a **light green 3** background.

   2. Negative information like warnings and API usage restrictions should be formatted as a **single-cell table** with a **light orange 3** background.

   It is recommended that you keep your infoboxes clean, concise and focused on a single topic. If you have information which may be useful to know but is not a critical part of the codelab instructions, then you should link to that information from the FAQ section rather than including it as an infobox.

7. Command-line Snippets

   It's often the case that you will have command-line instructions or log messages which are formatted with a monospaced font and have pre-defined whitespace. You can add these sections to your codelab by creating a **single-cell table** and making sure that all the text is formatted using the **Consolas font**.

8. Code Snippets

   Code snippets with syntax highlighting can be added by creating a **single-cell table** and making sure that all the text is formatted using the `Courier New` font.

   Any additional styles applied to the code will be overridden by the syntax highlighting. This means that you're free to use code highlighting Add-ons in your codelab doc but it

will have no effect on how the code is highlighted in the final codelab.

It is also strongly recommended that you provide a **Heading 3** header directly above your code snippet with the file name. This helps students keep track of where the code is coming from. The codelab framework also uses the file extension of the prior header as a clue for how to highlight the code.

It is also strongly recommended that you make your **Heading 3** header a hyperlink to the actual file if it is available on GitHub. A GitHub icon will automatically be added to the heading in such cases.

9. Frequently Asked Questions

   As the author of the codelab, you have developed and tested your code. You've probably run into all sorts of common issues or misconceptions. By linking to frequently asked questions, after each step where they often occur, you will reassure the students that they have everything they need to complete the codelab and avoid having to explain everything inline in your codelab. FAQs are easy to add. All the author needs to do is provide an unordered list of hyperlinks and preface it with a **Heading 3** header with the exact text: **Frequently Asked Questions**.

   Link icons will be added automatically for stackoverflow.com, developers.google.com, developers.android.com and support.google.com. All links will be configured to open in a new tab.

10. Download Buttons

    You can make it really easy to get started by including direct download buttons inline in your codelab. In order to add a button to your codelab, simply add a hyperlink and make sure that it is highlighted with a **dark green 1** background.

    Additionally, if the link text begins with the word "Download", a file download icon will be added to the button.

11. Per-step Time Estimates

    Many participants are not fully committed to completing a codelab when they start it. One of the ways that we can keep them in our codelab is by giving them accurate estimates about how much additional effort is required to complete the codelab at each step.

    In order to add this feature to your codelab, simply annotate each step in your codelab doc with a Duration: which uses **dark grey 1** text.

    That's it. The codelab framework will do everything else for you. If you forget to annotate a step with a duration, the default is 1:00. Also, if the last step of your codelab is just a

congratulations page, you should set the duration of that step to **0**.

12. Conditional Steps

    Sometimes it's useful to have different versions of a codelab for different environments. For example, you might have some steps that only apply to students who take the codelab in a classroom setting while other steps only apply to people who are following the instructions at their own pace online.

    The format is similar to the duration metadata. You simply specify one or more environments with an Environment: which uses **dark grey 1** text. The Duration and/or Environment fields, when present, should be followed by a blank line and should be set in normal text (not in Heading 1, lest they be considered part of the step title).

    If no environment metadata is specified, the default environment is "Web, Kiosk".

    When previewing your codelab, you can change environments using the &env=web or &env=kiosk parameters.

13. Fragment imports

    It is possible for a codelab to import another doc as a step fragment. For instance, it could be a set of setup instructions shared among multiple codelabs:

    [[**import** funny dog]]

    The contents of the funny dog document will be inserted in the codelab, replacing import instruction.

    The instruction syntax is:

    - start with [[(two square brackets)
    - followed by **import** keyword in bold
    - followed by a link to a doc
    - end with]] (two square brackets)

    Caveat: The imported doc is limited to content within a step (hence the term "fragment"). Including multiple steps, or even the step title/heading, within the imported doc is not supported.

14. Resumable Codelabs

    When a user returns to a codelab, they may be returning via a bookmark or a short-link posted online, it usually takes them to the first page of the codelab. In that case, the codelab remembers where they left off and asks them if they wish to resume where they

left off. This makes it easier for the user to jump back in and gives us more accurate analytics about how long users spend on each step.

This is simply part of the framework. There is nothing that you need to do as a codelab author to enable this feature.

15. Feedback Links

    At the bottom of every step of the codelab there is a link for reporting bugs. This link can be configured using the **Feedback Link** setting in your metadata table.

16. Inline Surveys

    NOTE: Surveys cannot be used to collect data that can individually, or in conjunction with other information from this site, help locate and identify a particular user or reveal their sensitive demographics information. Any data collected should be sufficiently anonymized and aggregated. Also, consider that the surveys can possibly send a numerical ID of the selections instead of the actual value itself. Apart from helping on the localization front, this can also help prevent from injecting obvious PII values into GA.

    As we've seen in previous years, participants consume our codelabs for a wide range of reasons. In order to give us some more insight into how different people consume codelabs, we can ask them some quick multiple choice questions in the early stages of our codelab.

    You can configure these short survey questions to ask whatever you think is relevant to your codelab. In order to include a survey question in your codelab, add a single-cell table with a **light blue 3** background. Format your question with the **Heading 4** paragraph style and provide an **unordered list** of choices.

    The participants' answers will automatically be added as custom variables in Google Analytics which can help you understand things like:

    - *What is the difference in completion rate between novices and experts?*
    - *What is the average time spent for people who wanted to write code vs. people who just wanted to read?*
    - *Is the bounce rate affected by the students' preference in IDE?*

    Of course, we need to be mindful of our participants' time and concentration and only ask a few key questions. It is *not* recommended to have a survey after each step.

17. What you'll learn

    Having a header 2 of "What you'll learn" followed by a bullet point list creates a list of check marks.

A title of "What we've covered" has the same effect.

18. YouTube Video Embeds

    Use a video tag like so `<video id="DWAinkJ54AP8"></video>` to embed a video uploaded to YouTube with the URL https://www.youtube.com/watch?v=DWAinkJ54AP8

Things to avoid

- **Footers:** Any characters included in the footer (beyond the default page number) result in parsing bugs. For this reason, page footers are not recommended.

WATERMARK WATERMARK WATERMARK